Name of Student: Husam Nour	**PRN:25070122086**
Semester: III	**Year: AY 2026-27**
Subject: Programming with Java	

ASSIGNMENT No: 26

TITLE: Write a Java program to create and execute multiple threads using the Runnable interface.

Problem Statement

Write a Java program to create and execute multiple threads using the Runnable interface.

```
class MyRunnable implements Runnable {
    private String name;
    MyRunnable(String name) {
        this.name = name;
    } public void run() { for (int i =
    1; i <= 5; i++) {
            System.out.println(name + " : " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
} public class A26 { public static void
main(String[] args) {
```

```
        MyRunnable r1 = new MyRunnable("Thread 1");
        MyRunnable r2 = new MyRunnable("Thread 2");
        MyRunnable r3 = new MyRunnable("Thread 3");
        Thread t1 = new Thread(r1);
        Thread t2 = new
        Thread(r2); Thread t3 =
        new Thread(r3);
        t1.start(); t2.start();
        t3.start();
    }
}
```

Learning Objectives

To create multiple threads using the Runnable interface.

Programme

1. Create a program using Runnable to run two tasks simultaneously.

```
class Task1 implements Runnable {
    public void run() { for (int i =
    1; i <= 5; i++) {
            System.out.println("Task 1: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
```

```
} class Task2 implements Runnable {
public void run() { for (int i = 1; i
<= 5; i++) {
            System.out.println("Task 2: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
} public class A26 { public static void
main(String[] args) {
        Task1 task1 = new Task1();
        Task2 task2 = new Task2();
        Thread t1 = new
        Thread(task1); Thread t2 =
        new Thread(task2);
        t1.start(); t2.start();
    }
}
```

2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

```
class BankAccount {
    int balance =
    1000;
    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println(Thread.currentThread().getName()
```

```
                + " deposited: Rs. " + amount);
        System.out.println("Current Balance: Rs. " + balance);
    }
    synchronized void withdraw(int amount)
        { if (balance >= amount) { balance
        -= amount;
            System.out.println(Thread.currentThread().getName()
                    + " withdrew: Rs. " + amount);
            System.out.println("Current Balance: Rs. " + balance);
        } else {
            System.out.println(Thread.currentThread().getName()
                    + " cannot withdraw. Insufficient balance!");
        }
    }
}
class DepositTask implements Runnable {
    BankAccount account;
    DepositTask(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
            account.deposit(500);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
class WithdrawTask implements Runnable {
    BankAccount account;
    WithdrawTask(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
            account.withdraw(300);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
public class A26 { public static void
    main(String[] args) {
        BankAccount account = new BankAccount();
        Thread t1 = new Thread(new DepositTask(account), "Deposit Thread");
        Thread t2 = new Thread(new WithdrawTask(account), "Withdrawal
        Thread") t1.start(); t2.start();
```

;

```
    }
}
```

Output

Write a Java program to create and execute multiple threads using the Runnable interface.

```
PS C:\CS-JAVA\28> javac A26.java
PS C:\CS-JAVA\28> java A26
Thread 2 : 1
Thread 1 : 1
Thread 3 : 1
Thread 2 : 2
Thread 1 : 2
Thread 3 : 2
Thread 2 : 3
Thread 3 : 3
Thread 1 : 3
Thread 2 : 4
Thread 3 : 4
Thread 1 : 4
Thread 3 : 5
Thread 2 : 5
Thread 1 : 5
PS C:\CS-JAVA\28>
```

1. Create a program using Runnable to run two tasks simultaneously.

```
PS C:\CS-JAVA\28> javac A26.java
PS C:\CS-JAVA\28> java A26
Task 2: 1
Task 1: 1
Task 1: 2
Task 2: 2
Task 1: 3
Task 2: 3
Task 1: 4
Task 2: 4
Task 1: 5
Task 2: 5
PS C:\CS-JAVA\28>
```

2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

```
PS C:\CS-JAVA\28> java A26
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1500
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1200
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1700
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1400
Deposit Thread deposited: Rs. 500
Current Balance: Rs. 1900
Withdrawal Thread withdrew: Rs. 300
Current Balance: Rs. 1600
PS C:\CS-JAVA\28> S
```

.

Exercise

1. Create a program where multiple threads print numbers simultaneously

```
class NumberThread extends Thread {
    public void run() { for (int i =
    1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + ": " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
public class A26 { public static void
    main(String[] args) {
        NumberThread t1 = new NumberThread();
        NumberThread t2 = new NumberThread();
        NumberThread t3 = new NumberThread();
        t1.setName("Thread
        1");
        t2.setName("Thread
        2");
        t3.setName("Thread
        3"); t1.start();
        t2.start();
        t3.start();
    }
}
```

```
PS C:\CS-JAVA\28> javac A26.java
PS C:\CS-JAVA\28> java A26
Thread 2: 1
Thread 3: 1
Thread 1: 1
Thread 2: 2
Thread 3: 2
Thread 1: 2
Thread 3: 3
Thread 1: 3
Thread 2: 3
Thread 3: 4
Thread 1: 4
Thread 2: 4
Thread 3: 5
Thread 2: 5
Thread 1: 5
PS C:\CS-JAVA\28>
```

2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously

```
class BankAccount {
    int balance =
    1000;
    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println(Thread.currentThread().getName()
                + " deposited: Rs. " + amount);
        System.out.println("Balance: Rs. " + balance);
    }
    synchronized void withdraw(int amount)
        { if (balance >= amount) { balance
        -= amount;
            System.out.println(Thread.currentThread().getName()
                    + " withdrew: Rs. " + amount);
            System.out.println("Balance: Rs. " + balance);
        } else {
            System.out.println("Insufficient balance!");
        }
    }
}
class DepositThread extends Thread {
    BankAccount account;
    DepositThread(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
```

```
            account.deposit(500);
```

```
        }
    }
}
class WithdrawThread extends Thread {
    BankAccount account;
    WithdrawThread(BankAccount account) {
        this.account = account;
    }
    public void run() { for (int i =
        1; i <= 3; i++) {
            account.withdraw(300);
        }
    }
}
public class A26 { public static void
    main(String[] args) {
        BankAccount account = new BankAccount();
        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new
        WithdrawThread(account); t1.setName("Deposit
        Thread"); t2.setName("Withdrawal Thread");
        t1.start(); t2.start();
    }
}
```

```
PS C:\CS-JAVA\28> javac A26.java
PS C:\CS-JAVA\28> java A26
Withdrawal Thread withdrew: Rs. 300
Balance: Rs. 700
Withdrawal Thread withdrew: Rs. 300
Balance: Rs. 400
Withdrawal Thread withdrew: Rs. 300
Balance: Rs. 100
Deposit Thread deposited: Rs. 500
Balance: Rs. 600
Deposit Thread deposited: Rs. 500
Balance: Rs. 1100
Deposit Thread deposited: Rs. 500
Balance: Rs. 1600
PS C:\CS-JAVA\28>
```

Conclusion

In conclusion, the Runnable interface in Java allows us to create and execute multiple threads to perform different tasks concurrently. It provides the run() method to define the task, while the Thread class is used to start and manage its execution. Using the Runnable interface improves code reusability and allows a class to extend another class while implementing multithreading. It is useful in applications such as banking systems, ticket booking, and online shopping, where multiple tasks need to execute concurrently.

Athread is the smallest unit of execution within a Java program. Multithreading enables multiple tasks to execute concurrently, improving application performance and responsiveness. Java provides the Thread class for creating threads by extending it and overriding the run() method. A thread passes through different lifecycle stages including New, Runnable, Running, Blocked/Waiting, and Terminated. The scheduler manages thread execution based on system resources. Methods such as start(), sleep(), join(), and interrupt() control thread behavior. Multithreading is commonly used in games, web servers, animation, and background processing applications